

CRSS-Python v1.0.0

A Full-Lifecycle Safety Standard for Python

CRSS = Certifiable Reliability & Safety Standard for Python

© 2025 Sofian Daghsen - All rights reserved

Distributed under CC BY-NC-ND 4.0

License: <https://github.com/sofiand-png/pycodereview/LICENSE-CRSS>

This whitepaper is provided “as is”, without warranties of any kind, including but not limited to merchantability, fitness for a particular purpose, or non-infringement. The author shall not be liable for any direct, indirect, incidental, consequential, or special damages arising from the use of this document or the CRSS-Python standard. Users are solely responsible for validating its applicability, correctness, and sufficiency within their own project, domain, and safety context.

Official Release and Publication Date: 22/12/2025

Status: Stable

1. Executive Summary

Python has historically been excluded from safety-critical domains such as automotive (ASIL-D), aerospace, industrial automation, and medical devices-not because its expressive power is insufficient, but because **its dynamic nature conflicts with the determinism, traceability, and evidence-based assurance** demanded by high-integrity systems.

The **CRSS Python Standard** (Code Review Safety Standard - Python Edition) provides a **unified safety framework** that transforms Python into a **predictable, enforceable, evidence-driven engineering environment** suitable for high-criticality software.

This whitepaper presents the **vision, design principles, and architecture** behind CRSS-Python v1.1.0. It does *not* replace the specifications; instead, it explains the motivation, conceptual models, and safety rationale that the rulebooks and governance processes implement.

Key capabilities introduced by CRSS-Python

- A **deterministic and constrained Python subset** validated through profiles, safety levels, and operational modes.
- A **Phase-Aware Rule Interpretation Model** that adapts enforcement to critical and non-critical code phases.
- A **comprehensive governance system** including compliance phases, safety evidence modeling (SCEM), and external assessments.
- A set of **Strict strengthening rules** that eliminate Python's notorious hazards (dynamic typing issues, unsafe reflection, GC nondeterminism, async cancellation, mutation traps, and more).
- A **tooling ecosystem model** enabling automated evidence generation and continuous enforcement.

Who this whitepaper is for

- Engineering teams building Python systems with safety or mission-critical requirements
- Technical leads, architects, and compliance officers
- Certification bodies and auditors
- Organizations considering Python for regulated environments
- Toolchain developers building CRSS-compatible analyzers or CI/CD integrations

CRSS-Python enables what was previously difficult or impossible: **safe, certifiable Python at industrial scale**.

2. Motivation

Python has become one of the world's most widely used languages across AI, automation, robotics, DevOps, scientific computing, and embedded control. Yet when systems approach safety-critical thresholds, Python is almost always removed in favor of C, C++, Rust, Ada, or proprietary DSLs.

Why?

Because high-integrity environments require:

- Deterministic runtime behavior
- Predictable memory and resource usage
- Traceable error handling
- Minimal dynamic side effects
- Exhaustive tool-based verification
- Evidence-based safety arguments
- Strict separation of critical vs. non-critical code

Python, in its common form, offers:

- Late binding
- Highly dynamic object models
- Non-deterministic garbage collection
- Global interpreter lock (GIL) interactions
- Unrestricted reflection
- Unpredictable async cancellation semantics
- Dynamic typing without guarantees

CRSS-Python is the architectural solution.

It provides:

- A precisely defined, enforceable subset of Python
- Configurable safety profiles
- Deterministic operational modes
- A complete evidence lifecycle
- A heavily restricted dynamic feature set
- Well-defined integration points for tooling and certification

The result is not “Python with many rules.”

It is **Python re-engineered** into a predictable, traceable, analyzable environment.

3. Industry Impact

The introduction of CRSS-Python changes the engineering calculus for safety-critical software:

3.1 Unlocking Python for critical systems

Organizations can now apply Python to domains where it was previously excluded:

- Perception pipelines with safety bottlenecks
- Supervisory control logic
- Configuration and validation layers
- Safety-adjacent ML integrations
- Deterministic data processing

- Highly monitored runtime components

CRSS makes this possible by **reducing ambiguity** and **enforcing deterministic subsets**.

3.2 Alignment with industry safety expectations

CRSS aligns with modern regulatory methodologies:

- ISO 26262 (ASIL)
- DO-178C (aerospace)
- EN 50128 / 50657 (rail)
- IEC 61508 (industrial)
- IEC 62304 (medical)

Rather than mimic these frameworks, CRSS maps Python semantics into **concepts compatible with auditor expectations**:

- Rule separation
- Strengthening mechanisms
- Evidence-based assurance
- Traceability
- Process maturity

3.3 Integration with modern development practices

CRSS is CI-first and tooling-first:

- Static analyzers
- Type checkers
- Linting engines
- Runtime instrumentation
- Evidence exporters
- CI/CD gating

This is a crucial advantage over older safety languages that lack contemporary developer workflows.

3.4 Organizational transformation

CRSS allows large organizations to:

- unify their Python guidelines under one formal standard,
- operationalize safety constraints through automation,
- integrate safety rules into linters and static analyzers,
- migrate safety-adjacent components into Python safely,
- reduce training overhead for developers.

3.5 Long-term certification trajectory

CRSS-Python doesn't attempt to rewrite industry certification. Instead, it provides:

- The **evidence structures**
- The **tooling requirements**
- The **compliance paths**

that certification bodies expect.

A standard like this is a key step in making Python acceptable in safety-critical engineering **without modifying the language** or creating proprietary subsets.

4. CRSS Overview

CRSS-Python is a **multi-component safety framework**, not a single document. Its strength comes from the integration of three pillars:

1. **The Specification Layer**
2. **The Governance Layer**
3. **The Tooling and Evidence Layer**

Together, these create a complete, enforceable safety ecosystem.

4.1 What CRSS Is

- A **safety and compliance standard** for professional Python engineering.
- A deterministic **rule and mode system** applied to Python code.
- A complete **evidence-based safety model**.
- A **process model** that ensures certification readiness.
- A **tooling model** that ensures automation and enforcement.

4.2 What CRSS Is Not

- Not a fork of Python.
- Not a new language or DSL.
- Not a general-purpose style guide.
- Not a static analyzer.
- Not tied to any particular framework or library.

CRSS is a **framework for achieving high integrity**, independent of application domain.

4.3 Architecture of the CRSS Standard

CRSS is composed of coordinated specifications and documents:

- **Profiles** (Core / Strict)
- **Safety Master Specification**
- **Mode & Safety Model**
- **Phase-Aware Rule Interpretation Model**
- **SCEM (Safety Case Evidence Model)**
- **Compliance Process Master**
- **External Assessment Process**
- **Tooling & Automation Specification**
- **Deployment / Release Management Models**

This modular architecture ensures each domain is independently maintainable yet conceptually unified.

4.4 Relationship Between Specifications, Processes, and Governance

CRSS integrates code-level rules with system-level governance:

- **Specifications** define “what must be true.”
- **Processes** define “how it becomes true.”
- **SCEM** defines “how it is proven true.”
- **Tooling** defines “how it is enforced automatically.”
- **External assessment** defines “how others verify it is true.”

This alignment ensures CRSS is not just a rulebook-it is a complete safety system.

4.5 CRSS Design Principles

4.5.1 Determinism

CRSS enforces deterministic execution semantics for all safety-relevant Python.

4.5.2 Predictability

Every construct allowed in Strict Mode has predictable runtime behavior.

4.5.3 Constrained Dynamism

Dynamic Python features are tightly controlled, isolated, or prohibited based on safety level.

4.5.4 Evidence-Based Assurance

Every claim in the system must be backed by SCEM artifacts.

4.5.5 No Hidden Runtime Behavior

Imports, closures, async tasks, exceptions, and GC interactions must behave *explicitly*, not implicitly.

5. Profiles, Safety Levels, and Modes

Modern safety engineering standards often rely on hierarchical constraint systems: a combination of functional criticality, operational integrity, and design-time constraints. CRSS follows this philosophy with a **three-axis safety structure**:

- **Profiles** - define rule strictness
- **Safety Levels** - define impact severity
- **Modes** - the combined operational enforcement tier (Profile × Level)

Together, these form the **CRSS Enforcement Model**.

5.1 Profiles

Profiles define **how strict the rulebook is**. They do not describe the project's functional safety level—that is the role of Safety Levels. Rather, Profiles describe the deterministic and behavioral constraints desired for the codebase.

CRSS defines two Profiles: **Core** and **Strict**.

5.1.1 Core Profile

The **Core Profile** defines the baseline deterministic programming subset of Python:

- Dynamic features are allowed but constrained
- Reflection and metaprogramming are allowed with limitations
- Indirect dynamic behavior requires explicit justification
- Runtime patterns that introduce implicit behavior are discouraged
- Code must remain testable, analyzable, and predictable

Core is the minimum level required for CRSS compliance.

5.1.2 Strict Profile

The **Strict Profile** defines a **highly constrained safety subset** of Python.

It enforces:

- No unsafe dynamic features
- Restricted reflection
- Strict object lifecycle determinism
- Strongly defined exception handling
- Fully deterministic async and concurrency
- Explicit resource lifecycle constraints
- Disallowed implicit conversions
- Disallowed late binding and mutation traps

Strict removes the ambiguity that safety-critical systems cannot tolerate.

5.1.3 Relationship Between Core and Strict

Strict is not an alternative to Core.

It is a **superset**:

Strict Profile $\supset$ Core Profile

Any rule that applies to Core also applies to Strict, while Strict adds:

- strengthened versions of certain Core rules
- additional safety guarantees
- stronger restrictions
- tighter constraints on dynamic behavior

Strict is used for high-criticality software and for teams who want maximum enforcement discipline.

5.2 Safety Levels

Safety Levels describe **how severe a failure would be**.

CRSS defines three levels:

- **A - Highest criticality**
- **B - Elevated risk**
- **C - Low severity / non-critical**

These levels align conceptually (not formally) with well-known safety frameworks:

- **ASIL-D $\approx$ Level A**
- **ASIL-C/B $\approx$ Level B**
- **ASIL-A / Quality $\approx$ Level C**

They are designed to serve as **inputs to the Mode System**.

5.2.1 Level A - Highest Criticality

Failure may cause catastrophic impact.

Rules enforced at Level A must:

- ensure deterministic behavior
- eliminate memory and execution ambiguity
- fully constrain dynamic operations
- ensure complete traceability
- forbid nondeterministic concurrency patterns

5.2.2 Level B - Elevated Risk

Failure may contribute to system-level degradation but not catastrophic failure.

Constraints:

- strict for safety-relevant components
- moderate for safety-adjacent logic
- controlled dynamic behavior permitted

5.2.3 Level C - Lowest Criticality

Failure poses limited safety consequence.

Constraints:

- Core suffices
- dynamic features allowed with care
- less stringent error-handling requirements

Level C does *not* remove CRSS rules.

It simply loosens constraints where justified.

5.3 Modes: The Combined Enforcement Model

A **Mode** is simply:

Mode = Profile × Safety Level

This creates six possible Modes:

Profile	Level	Mode Name
Core	C	Core-C
Core	B	Core-B
Strict	C	Strict-C
Strict	B	Strict-B
Strict	A	Strict-A

Note: Core-A is forbidden.

Modes determine:

- which rules apply
- which strengthened rules activate
- which dynamic features are allowed
- what level of determinism is required
- which evidence must be produced

5.3.1 What Is a Mode?

A Mode is an **enforcement context**.

It configures:

- rule applicability
- severity interpretation
- tool behavior
- evidence requirements

5.3.2 Strict-A: The Zero-Tolerance Mode

Strict-A is the **highest enforcement tier**.

It represents:

- Fully deterministic behavior
- All dynamic features restricted or prohibited
- Strict object lifetime guarantees
- No runtime surprises
- Heavy SCEM evidence expectations

Strict-A is suitable for Level A safety functions in aerospace, automotive, medical, and robotics.

5.3.3 Mode Propagation (Dependency Escalation)

If a component depends on a stricter Mode, it **inherits the stricter Mode**.

Example:

- Component X is Strict-A
- Component Y is Core-B
- X depends on Y

Then Y must be elevated to *at least* Strict-A.

No dependency may reduce the safety posture of a stricter component.

5.4 Deterministic Behavior per Mode

Modes define how deterministic behavior is enforced.

5.4.1 Allowed and Disallowed Patterns

Example - late-bound closures:

- Allowed in Core-C
- Allowed with restrictions in Core-B
- Forbidden in Strict modes

5.4.2 Rule Strengthening

Rules may become stricter depending on Mode.

E.g., a Core rule that is “SHOULD NOT” may become “MUST NOT” in Strict-A.

5.4.3 Profile-Level Enforcement Logic

Tools interpret Modes as a **hierarchy**:

Strict-A > Strict-B > Strict-C > Core-B > Core-C

The highest Mode wins during evaluation.

6. Phase-Aware Rule Interpretation

CRSS introduces a novel concept: **rules that behave differently depending on the phase of the code.**

In modern systems, not all code paths are equal.

Some are **critical**, others are **supportive**, and some are **purely auxiliary**.

Applying identical constraints everywhere reduces efficiency and increases false violations.

CRSS solves this with the **Phase-Aware Model**.

6.1 Purpose of Phase-Aware Semantics

Safety integrity must apply **only where required**.

Phase-aware interpretation enables:

- higher determinism where needed
- flexibility where safe
- better performance when appropriate
- clearer safety boundaries

This mirrors real-world engineering.

6.2 Code Phases in CRSS

CRSS defines two conceptual execution phases.

6.2.1 Critical Phase (@critical)

A code path is critical when:

- it contributes directly to a safety function
- its timing, determinism, or failure affects system safety
- its behavior influences decision-making in safety context

6.2.2 Non-Critical Phase

Any code path that does not meet @critical requirements.

May be:

- I/O
- logging
- analytics
- auxiliary tasks
- user interface logic

6.2.3 Transition Points

A critical phase may call a non-critical component only if:

- deterministic behavior is preserved
- evidence justifies the transition
- the called component meets deterministic requirements

6.3 How Rules Behave Across Phases

Each rule with phase-aware semantics defines **different severity behavior** in each phase.

6.3.1 Severity Mapping

- INFO
- WARN

- ERROR
- BLOCKER

Example:

- Dynamic attribute creation → WARN in non-critical, BLOCKER in critical phases.

6.3.2 Tooling Interpretation

Phase-aware rules require tools to:

- recognize annotated phases
- apply different severity
- produce phase-specific evidence

6.3.3 Runtime Interpretation

Critical code must exhibit:

- predictable GC interactions
- deterministic async behavior
- controlled exceptions
- strict resource handling

Non-critical paths may relax constraints.

6.4 Examples

6.4.1 Logging in Critical Code

- Allowed but must not modify safety state
- Must not block or raise unexpected exceptions

6.4.2 I/O Operations

- Forbidden in Strict-A critical phase
- Allowed in non-critical with restrictions

6.4.3 Blocking Operations

- Disallowed in all Strict critical paths

6.4.4 Dynamic Features

- Reflection allowed in non-critical
- Nearly always blocked in critical

6.5 Interaction with Profiles and Modes

The Phase-Aware Model layers with Modes cleanly:

- **Mode** determines rule strictness
- **Phase** determines rule severity

- Combined, they give deterministic, context-aware enforcement

This avoids “one size fits all” safety policy.

7. CRSS Rule Architecture

CRSS rules encode all safety restrictions, allowed patterns, and prohibited constructs. Unlike traditional linters, CRSS rules include:

- metadata
- safety rationale
- examples
- profile applicability
- phase-aware interpretation
- SCEM traceability hooks

7.1 Rule Structure

Each rule includes:

7.1.1 Rule ID Scheme

CRSS-<chapter>.<section>.<rule>

7.1.2 Category

Example categories:

- Control Flow
- Dynamic Features
- Types & Interfaces
- Concurrency
- Memory & Resources
- I/O
- Security

7.1.3 Type

- Static
- Dynamic
- Process

7.1.4 Profiles Applicability

Defines applicability to Core vs Strict.

7.1.5 Scope

- `all_code`
- phase-aware
- module-level
- function-level
- class-level

7.1.6 Rationale

The safety justification behind the rule.

7.1.7 Examples

Every rule includes:

- compliant code
- non-compliant code

Rule clarity is essential for tool builders.

7.2 Strengthening Model

7.2.1 Why Strengthening Exists

Python is dynamic by design.

Strengthening limits unsafe constructs based on required deterministic behavior.

7.2.2 Examples

- Late-bound closures: discouraged in Core, forbidden in Strict
- Mutable default arguments: warned in Core, blocked in Strict
- Async cancellation: restricted in Core, deterministic cleanup required in Strict
- Native extensions: allowed via adapters only in Strict

7.2.3 Strict-A Specifics

Strict-A mandates:

- total determinism
- deterministic object lifecycle
- static-type constraints
- static import dependency graphs
- fully deterministic exception flow

7.3 Python-Specific Rule Families

Python introduces unique hazards.

CRSS addresses each family of hazards with targeted rule sets.

7.3.1 Dynamic Features

- restricted attribute creation
- forbidden monkeypatching
- restricted eval/exec
- controlled class mutation

7.3.2 Reflection

Reflection remains allowed but restricted to predictable patterns.

7.3.3 Type System

- require explicit typing
- no ambiguous duck-typing in Strict-A
- clear interface boundaries

7.3.4 Memory and GC Rules

- deterministic resource cleanup
- predictable object lifetime
- GC nondeterminism fully controlled

7.3.5 Concurrency

- task supervision
- deterministic cancellation
- no unmanaged background tasks

7.3.6 Numeric Determinism

- NaN/Inf handling rules
- deterministic rounding
- required Decimal usage in Strict-A for financial values

7.3.7 I/O and Timing

- no blocking calls in critical Strict-A code
- deterministic timeouts
- explicit failure behavior

8. Python Safety Hazards and Mitigations

Python's expressive power creates hazards that do not exist in strongly deterministic languages.

CRSS directly targets these hazards.

8.1 The Python Safety Problem Space

The hazards come from:

- dynamic typing
- reflection
- late binding
- mutability
- GC nondeterminism
- async unpredictability
- dependency graph ambiguity
- hidden side effects

CRSS eliminates or constrains each hazard.

8.2 Key Hazard Areas

8.2.1 Mutable Default Arguments

A classic Python pitfall that leads to shared state leaks.

CRSS Strict-A: **Prohibited**

CRSS Core: **Warn & discourage**

8.2.2 Late-Bound Closures

CRSS eliminates late binding traps through explicit binding requirements.

8.2.3 Identity vs Equality Misuse

CRSS prohibits misuse of `is` for value comparison.

8.2.4 Non-Deterministic Garbage Collection

GC-induced pauses must not affect critical code.
Rules mandate deterministic cleanup.

8.2.5 Import-Time Side Effects

Imports must be idempotent and deterministic.

8.2.6 Dynamic Object Shapes

CRSS restricts shadowing, monkeypatching, and structure mutation.

8.2.7 Async Cancellation

CRSS requires:

- explicit cleanup
- deterministic teardown
- supervised task lifecycles

8.2.8 Exceptions as Control Flow

Strict modes require explicit error-handling boundaries.

8.2.9 Unsafe Reflection

Allowed only with strict controls.

8.2.10 Native Extension Boundaries

FFI must be routed through controlled adapters.

8.3 CRSS Mitigation Strategy

8.3.1 Static Restrictions

Prevent entire classes of failures.

8.3.2 Deterministic Subset

Restricts Python to predictable patterns.

8.3.3 Tool-Assisted Enforcement

Machine-verifiable rule compliance is mandatory.

8.3.4 Runtime Guarantees

Async, cancellation, GC, and object lifetimes behave deterministically in Strict-A.

9. Safety Case Evidence Model (SCEM)

The foundation of verifiable, certifiable Python safety.

The **Safety Case Evidence Model (SCEM)** is the backbone of CRSS-Python. While rules define what constitutes safe and deterministic behavior, **SCEM provides the mechanism to prove that the rules have been satisfied.**

Modern certification processes for software systems expect:

- clear claims
- explicit evidence
- structured traceability
- automated verification
- objective validation

SCEM fulfills these expectations for Python systems.

9.1 Purpose of SCEM

SCEM defines:

- **What evidence must be produced**
- **How evidence is structured**
- **How evidence relates to CRSS rules, processes, and Modes**
- **How completeness and sufficiency are evaluated**
- **How auditors and external assessors can trace claims to validation artifacts**

This transforms compliance from a subjective process to an **objective, machine-verifiable system.**

9.2 Structure of SCEM

SCEM is a **graph-based evidence model**, consisting of:

- **Domains** (logical categories)
- **Evidence nodes** (atomic artifacts)
- **Evidence links** (traceability relationships)
- **Validation rules** (conditions that must be satisfied)

Together, they form a directed acyclic graph (DAG) representing the assurance argument.

9.2.1 Evidence Domains

Domains typically include:

- **Static Analysis Evidence**
- **Dynamic/Runtime Evidence**
- **Process Evidence**
- **Toolchain Evidence**
- **Release & Deployment Evidence**
- **Exception Justification Evidence**

Each domain groups evidence of similar purpose but different origin.

9.2.2 Evidence Nodes

Nodes represent individual artifacts, such as:

- a static analysis violation report
- a Mode configuration table
- runtime logs validating deterministic cleanup
- exception propagation maps
- unit test coverage summaries
- dependency graphs
- Phase-aware interpretation outputs
- rule violation justifications
- tool qualification reports

Nodes carry metadata:

- origin (tool/process/engineer)
- timestamp
- mode applicability
- validation state
- hash (for immutability)

9.2.3 Evidence Links

Links express:

- **satisfaction** (“A satisfies B”)
- **derivation** (“A is derived from B”)
- **dependency** (“A depends on B”)

- **coverage** (“A covers B”)
- **verification** (“A verifies B”)

This allows SCEM to represent rich traceability such as:

- A rule → is enforced by → Static analyzer output
- A test → covers → Safety requirement
- A justification → addresses → Violation exception

9.2.4 Evidence Validation Rules

Each evidence domain has rules that determine:

- how evidence is validated
- whether it is complete
- whether conflicts exist
- whether contradictory claims occur

Example:

- All Strict-A evidence nodes must be machine-generated, not manually written.
- Critical-phase evidence must include runtime coverage artifacts.
- Native extension use must provide boundary verification evidence.

9.3 SCEM Completeness Criteria

A SCEM instance is **complete** when:

- Every CRSS claim has at least one validating evidence node
- Every rule has a corresponding enforcement report
- Every component has a Mode assignment
- All exceptions have formal justifications
- All runtime behaviors declared deterministic have runtime validation evidence
- All dependencies have Mode propagation verification

Completeness is mechanically verifiable.

9.4 SCEM and Profile/Mode Mapping

Evidence requirements depend on Mode:

- Strict-A → requires full static + dynamic + process evidence
- Strict-B → requires static + process evidence
- Core-B/C → require lighter evidence sets

SCEM guarantees that Mode selection is not simply configuration-it influences required assurance depth.

9.5 SCEM Examples

Example 1 - Mutable default arguments violation

Evidence includes:

- Static analyzer detection event
- Developer justification (if non-critical)
- Unit test demonstrating isolation
- Mode-based severity classification

Example 2 - Async task supervision in Strict-A

Evidence includes:

- Explicit supervision tree
- Runtime logs confirming deterministic cancellation
- Failure injection tests
- Phase-aware severity mapping

Example 3 - Native extension usage

Evidence includes:

- Adapter design documentation
- Python-to-native boundary contract
- Integration stress tests
- Failure-mode analysis

9.6 SCEM in Certification

SCEM is the primary artifact reviewed in:

- Internal audits
- External assessments
- Pre-certification reviews
- Configuration baseline evaluations
- Change impact analysis

It provides the **engineer-to-assessor handshake** that Python has historically lacked.

10. Compliance Process (Phases 0–5)

From initial project enrollment to certification readiness.

CRSS defines a structured lifecycle for compliance.

It ensures teams progress systematically from project kickoff to certification readiness, with quantitative checkpoints and required outputs.

The process has **six phases**.

10.1 Overview of the Compliance Lifecycle

Each phase introduces:

- new requirements
- new evidence expectations

- new verification steps
- new enforcement depth

Certification readiness occurs only after Phase 5.

10.2 Phase 0 - Project Enrollment

Artifacts:

- initial Mode assignment
- preliminary dependency graph
- declared safety-level boundaries
- tooling selection proposal
- CRSS readiness scoring (baseline)

Outcome:

- Project registered as CRSS-monitored.

10.3 Phase 1 - Baseline Establishment

Artifacts:

- dependency graph validation
- Mode propagation
- baseline rule violations
- toolchain bootstrapping
- developer onboarding documentation

Outcome:

- Project baseline frozen.
- All teams aligned on CRSS constraints.

10.4 Phase 2 - Static Analysis

Artifacts:

- complete rule violation report
- rule exceptions + formal justifications
- strengthened rule activation verification
- type system validation (e.g., mypy)

Outcome:

- Codebase achieves “static compliance baseline.”

This is often the longest phase.

10.5 Phase 3 - Dynamic & Behavioral Analysis

Artifacts:

- runtime behavior logs
- GC stability results
- async cancellation behavior evidence
- exception handling coverage
- deterministic timing evidence
- failure injection results

Outcome:

- Behavioral correctness validated.
- Determinism confirmed for Mode expectations.

10.6 Phase 4 - Evidence Integration

Artifacts:

- complete SCEM instance
- evidence validation report
- safety argument summary
- dependency impact analysis
- Mode compliance revalidation

Outcome:

- Evidence ready for external review.

10.7 Phase 5 - Final Compliance Decision

Artifacts:

- final compliance report
- certification readiness declaration
- unresolved violation exceptions (optional)
- release candidate CBM

Outcome:

- Project qualifies for external assessment.

10.8 Compliance Artifacts

Examples:

- Full SCEM
- Compliance reports
- Exception lists
- Tool outputs
- Mode tables
- Dependency evidence maps

10.9 Transition Criteria

Phase transitions are **explicit** and **reviewed**.

No phase can be skipped.

11. External Assessment Process (EAP)

Independent validation for safety compliance.

The External Assessment Process (EAP) ensures that compliance is not self-declared; it is independently validated by a qualified external entity.

11.1 Purpose of EAP

EAP provides:

- independent verification
- unbiased scrutiny
- industry-grade validation
- regulatory-aligned assessment

It is the final gate before certification in many industries.

11.2 What Assessors Evaluate

Assessors review:

11.2.1 SCEM Completeness and Consistency

Every claim must be fully evidenced.

11.2.2 Tooling Outputs

Assessors check:

- static analysis reports
- runtime logs
- phase-aware rule interpretations

11.2.3 Rule Violations & Exceptions

Each exception must include:

- justification
- Mode compliance
- test coverage
- mitigation steps

11.2.4 Mode Consistency

Assessor verifies:

- Mode inheritance
- dependency Mode propagation
- justification of requirements

11.3 Assessment Workflow

1. Intake of SCEM and compliance documents
2. Automated validation check
3. Manual review of high-risk sections
4. Clarification rounds with engineers
5. Evidence challenge / counterevidence
6. Final validation decision

11.4 Assessment Report Format

The report includes:

- summary of findings
- compliance score
- violation severity distribution
- unresolved exceptions
- recommendations

11.5 Certification Outcomes

Possible outcomes:

- Fully Approved
- Approved with Conditions (**must resolve findings**)
- Not Approved

Certification is based on **objective, evidence-driven criteria**.

12. Tooling & Automation

Automation is mandatory, not optional.

CRSS requires that safety processes be **automated** wherever possible. This reduces human error and ensures consistent enforcement across large codebases.

12.1 Tool Capability Levels (TCL)

TCL defines the maturity of a tool:

- **TCL-1** - Basic analysis
- **TCL-2** - Deterministic behavior modeling
- **TCL-3** - Phase-aware interpretation
- **TCL-4** - Evidence export & SCEM integration
- **TCL-5** - Full automation with CI/CD blockers

12.2 Tool Compliance Attributes (TCA)

Tools must provide:

- reproducibility
- auditability
- determinism
- machine-readable outputs
- secure configuration

12.3 Static Analysis Tools

Must enforce:

- Core and Strict rules
- strengthened rules
- phase-aware rules
- dependency graph verification
- rule exception justification pipelines

12.4 Runtime Monitoring Tools

Must provide:

- async behavior analysis
- GC determinism measurements
- phase separation validation
- critical-path timing evidence

12.5 Evidence Export Requirements

Tools must output:

- JSON/graph representations
- SCEM node definitions
- traceability metadata
- Mode applicability

12.6 CI/CD Integration

CRSS requires that:

- compliance checks run in CI
- SCEM is updated automatically
- release gates block unsafe builds
- Mode tables are validated continuously
- dependency maps stay in sync

Automation is how CRSS ensures *continuous integrity*, not just manual integrity.

13. Deployment & Release Model

Ensuring deterministic and safe release pipelines.

Deployment is often one of the weakest points in safety-critical software. CRSS acknowledges this by defining a **deterministic deployment and release architecture**, so the safety integrity verified during development is not compromised during packaging, distribution, or runtime updates.

13.1 Deterministic Deployment Principles

CRSS mandates that deployment must be:

- **Deterministic:**
No uncontrolled variability across machines or environments.
- **Reproducible:**
Build outputs must be exactly reproducible from the same inputs.
- **Traceable:**
Every artifact must trace back to a specific source, commit, and Mode.
- **Immutable:**
Released packages must not mutate or depend on mutable external state.
- **Version-Locked:**
Dependencies must be explicitly declared and pinned.

These principles eliminate “environment drift,” a major source of failures in Python ecosystems.

13.2 Configuration Baseline Model (CBM)

The **CBM** defines the exact state of a software configuration for deployment.

It includes:

- Python version
- CRSS version
- Mode assignments
- Dependency versions
- Tool versions
- Static analysis configuration
- Runtime environment settings
- Release build metadata
- Test suite and coverage metadata
- Artifact signatures

A CBM is:

- **Immutable** once published
- **Verifiable** through hashes and signatures

- **Comparable** for detecting drift

CBM is essential for both internal and external audits.

13.3 Release Qualification

Before a release is accepted, it must:

1. Pass full CI/CD pipelines
2. Produce a SCEM snapshot
3. Pass Mode inheritance validation
4. Produce zero-blocker rule violations
5. Have justified accepted deviations
6. Pass runtime determinism validations
7. Include deployment-time safety checks

If any step fails, release qualification fails.

Strict-A releases are the most demanding-they require:

- complete SCEM
- deterministic timing evidence
- exception propagation mapping
- GC behavior logs
- async supervision evidence

13.4 Backward-Compatible Upgrades

CRSS deployment processes ensure that:

- Dependency upgrades must not violate Modes
- API changes must be validated against interface contracts
- Behavior compatibility must be checked across all Levels
- Rollback paths must exist for all critical deployments

Upgrading a Strict-A safety function requires:

- impact analysis
- regression evidence
- revalidation of strengthened rules
- verification of deterministic behaviors

13.5 Drift Prevention

Deployment drift must be prevented at all layers:

- dependency drift
- environment drift
- tool version drift
- configuration drift
- rule activation drift

CRSS enforces drift prevention through:

- CBM signatures
- CI/CD validation
- SCEM consistency checks
- Mode mapping verification
- Release artifact validation

No system can be considered safe if it drifts from its verified configuration.

14. Supported Python Versions

CRSS v1.1.0 officially supports:

Python 3.9.x → Python 3.12.x (inclusive)

Every minor branch within this range is supported.

This includes:

- **3.9.x**
- **3.10.x**
- **3.11.x**
- **3.12.x**

Notably NOT included:

- Python 3.13 (runtime and semantics not yet stable for CRSS)
- Any Python < 3.9 (missing modern features necessary for determinism and typing guarantees)

14.1 Version Range Rationale

CRSS selects versions based on:

Language Stability

Python 3.9–3.12 offer stable:

- type system
- async semantics
- collections behavior
- GC behavior
- interpreter-level guarantees

Deterministic Behavior

Newer interpreters provide more predictable:

- object lifecycle
- garbage collection
- scheduling
- async task supervision

Tool Compatibility

Static analyzers and type checkers have full compatibility for 3.9–3.12.

Security Support

All supported versions receive security updates.

14.2 Version Policy

CRSS follows a strict version lifecycle:

- **Support window:** 3 major CPython releases
- **Grace period:** 12 months for transition
- **End-of-life:** After 12 months without security fixes

Strict-A deployments must upgrade within the grace window.

14.3 Future Python Version Support

Python 3.13+ will be supported once:

- async semantics stabilize
- C API compatibility is fully validated
- tools adopt the new features
- deterministic behavior guarantees are confirmed

CRSS intentionally delays adoption until safety integrity can be ensured.

15. Architecture & Structure

CRSS-Python enforces structural safety as a first-class property. Architectural safety ensures that code organization, imports, inheritance, module boundaries, and component interactions cannot undermine critical safety guarantees. Architecture must be stable, hierarchical, traceable, and free from hidden coupling or nondeterministic flows.

15.1 Import Layering Rules

CRSS defines a strict, one-directional layering model:

- High-criticality modules may NOT depend on lower-integrity modules unless promoted via MAR
- Low-criticality modules may depend on higher-criticality modules
- Cyclic imports are forbidden
- Critical code may only import other critical-safe modules
- Non-critical modules must never leak I/O or state into critical paths

This protects critical logic from contamination and nondeterministic interactions.

15.2 Inheritance Safety Model

CRSS enforces controlled inheritance:

- Only single-level inheritance is allowed (depth of 1)
- Inheriting from a Level-A class elevates the subclass to Level-A
- Multiple inheritance is forbidden
- Inheritance chains may not introduce nondeterministic behavior
- Base classes for critical logic must be stable, well-documented, and free of side-effects

This prevents untraceable behavior that often arises from Python's flexible OOP mechanisms

15.3 Safety-Level Propagation Across Architecture

Safety levels propagate across architecture as follows:

- If module M_A (Level A) imports module M_B (Level B), then M_B becomes Level A for verification
- Classes inherit the stricter safety level of their ancestors
- Functions imported into Level A contexts must be verified as Level A

15.4 Component Boundary Isolation

CRSS defines strict boundaries between system components:

- Decision logic (critical) is isolated from operational logic (non-critical)
- Safety-critical computation must not cross module boundaries that include I/O
- Shared state is forbidden; explicit data transfer interfaces must be used
- All communication must be validated, bounded, and deterministic

This minimizes coupling and enhances traceability for certification.

15.5 Microservice Safety Design

For distributed systems, CRSS requires:

- deterministic, schema-bound API interactions
- retries with bounded backoff
- validated request/response payloads
- network-failure-tolerant design
- service timeouts and circuit breakers
- no critical logic distributed across network calls

Critical safety decisions MUST occur locally to guarantee determinism.

15.6 Safe Dataflow Architecture (Diagram)

----- Non-Critical I/O Layer ----- • Input adapters • Network clients • Device interfaces • File/db access

↓ sanitized data

----- Validation & Preprocessing ----- • schema validation • bounds checking • sanitization • safe parsing

↓ bounded, validated data

----- Critical Decision Layer ----- • deterministic computation •
allocation-free logic • GC disabled • pure functions

↓ decisions

----- Non-Critical Output Layer ----- • logging • networking •
publishing • storage

This ensures structural integrity even when components originate at different safety levels.

15.7 Avoiding Hidden Coupling

CRSS prohibits:

- global mutable state
- singletons
- implicit initialization logic
- thread-local state affecting critical logic
- side-effects during import
- magic dynamic dispatch

All behavior must be explicit, traceable, and reviewable.

15.8 Architectural Review Requirements

All CRSS projects must conduct formal architectural reviews, including:

- dependency graphs
- module boundary audits
- safety propagation mapping
- inheritance chain validation
- criticality boundary definition
- import-layer compliance
- I/O isolation validation

Architecture must be stable and documented for certification.

15.9 Summary of Structural Safety Guarantees

CRSS delivers:

- clean layering
- deterministic dataflow
- bounded interactions
- tightly controlled imports
- safe inheritance
- explicit component boundaries
- isolation of critical decision logic

These architectural guarantees form the backbone of certifiable Python systems.

16. Network, Microservices & Distributed Safety

16.1 Distributed Systems as a Safety Risk

Distributed systems introduce nondeterminism from latency, bandwidth, network jitter, packet loss, remote timeouts, and external system failures. CRSS-Python treats all network interactions as

non-deterministic by default and strictly forbids placing critical safety logic across network boundaries.

16.2 Deterministic Network Interaction Model

CRSS enforces:

- Network communication **MUST NOT** occur inside critical phases
- All network I/O must be bounded by explicit timeouts
- Retry mechanisms must use capped exponential backoff
- No unbounded waiting or blocking on remote calls
- All inputs from remote systems must undergo validation

This ensures network behavior can never jeopardize deterministic safety computation.

16.3 Microservice Safety Design

CRSS microservice safety rules:

- Critical decisions **MUST** be made locally
- Remote calls may only influence non-critical preprocessing
- All remote APIs must use fixed schemas
- Validation is mandatory before data enters any safety-relevant component
- No implicit state sharing or reliance on remote freshness guarantees

This prevents safety-critical logic from depending on the reliability of networked systems.

16.4 Safe API Schemas

All remote API inputs must:

- be schema validated (JSON-Schema, XML schema, protobuf, etc.)
- enforce strict types
- enforce bounded size
- reject unknown fields
- define deterministic fallbacks for missing/invalid data

The safety of the system is directly tied to the strictness of data schemas.

16.5 Handling Partial Failures

Distributed systems fail partially and asymmetrically. CRSS enforces:

- fail-safe defaults on remote timeouts
- circuit breakers for unstable services

- no infinite retries
- fallback decision logic (local, deterministic)
- strict separation of degraded vs normal modes

Safety relies on predictable, bounded error behavior.

16.6 Deterministic Distributed Architecture (Diagram)

----- Non-Critical Network Layer ----- • HTTP/GRPC clients • SFTP/FTP • Message brokers • Socket interfaces

↓ validated and bounded

----- Sanitization & Preprocessing ----- • schema validation • size constraints • sanitization • transformation

↓ safe data

----- Critical Decision Layer ----- • no network • deterministic logic • pure computation

↓ decision

----- Non-Critical Output Layer ----- • publish result • send API response • store to DB

16.7 Timeout, Retry & Backoff Safety Rules

CRSS requires:

- fixed maximum timeout
- bounded retry count
- exponential backoff with ceiling
- fallback error paths
- logging of remote instability

This ensures no distributed behavior can cause uncontrolled delays.

16.8 Isolation of Safety-Relevant Logic

Safety logic must always be:

- Local
- Deterministic
- independent from external timing
- independent from the state or availability of remote systems

Only non-critical preparatory or follow-up processing may span distributed nodes.

16.9 Networking Safety Review Checklist

A CRSS system must verify:

- critical logic contains zero network operations
- network inputs are schema-validated
- retries and timeouts are bounded
- microservice boundaries match safety levels
- circuit breakers and fallback logic are implemented
- no safety decision depends on remote freshness

These conditions ensure distributed safety integrity.

16.10 Summary of Distributed Safety Constraints

CRSS delivers a deterministic, bounded, and certifiable model for microservice and network-heavy architectures. Critical logic remains fully isolated from remote failure modes, enabling ASIL-D and SIL-3 supervisory safety behavior even in modern distributed software ecosystems.

17. Conclusion

CRSS-Python v1.0.0 represents a major milestone:

a fully engineered, deterministic, traceable, and certifiable Python ecosystem.

It enables something previously considered unattainable:

- Python in high-integrity environments
- Python in deterministic systems
- Python in safety pipelines
- Python that regulators can audit
- Python with quantified assurance and structured evidence

CRSS delivers:

- A restricted but powerful deterministic subset of Python
- A comprehensive rule system
- A multi-axis safety architecture (Profiles, Levels, Modes)
- A dynamic, context-aware enforcement model
- A complete evidence graph (SCEM)
- A structured compliance lifecycle
- Automated tooling integration
- A deterministic deployment model

This whitepaper provides the rationale and conceptual framework behind the specifications. The specifications enforce the rules., the governance model ensures they are followed, the tooling ecosystem ensures they remain followed. And the evidence model proves it.

Together, they form a **complete safety and compliance ecosystem.**

CRSS-Python is not simply a set of guidelines,
it is **a pathway toward certifiable Python software in a safety-critical world.**